UNIVERSIDADE FEDERAL DE SERGIPE

# Programação dinâmica em grafos

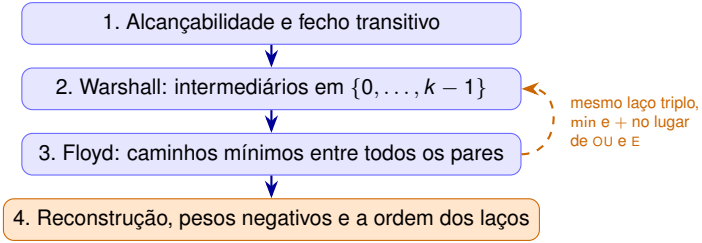
## Os algoritmos de Warshall e de Floyd

Prof. Dr. André Yoshiaki Kashiwabara

Departamento de Computação — Universidade Federal de Sergipe

4 de novembro de 2026

# O que você vai aprender hoje




Na LCS, o subproblema encolhia as *sequências* (prefixos). Hoje o que encolhe é o **conjunto de vértices** que um caminho pode atravessar.

# Fecho transitivo

# Por que perguntar quem alcança quem?

## Problema:

Para *todo* par $(i, j)$ de vértices de um digrafo, existe caminho de $i$ até $j$?

- Pré-requisitos: cursar AED I libera, direta ou indiretamente, quais disciplinas?
- Compilação: mudar um módulo obriga a recompilar quais outros?
- Bancos de dados: consultas recursivas (“quem é ancestral de quem”).

## Solução:

Uma busca (DFS ou BFS) a partir de cada vértice resolve em $O(n(n + m))$.

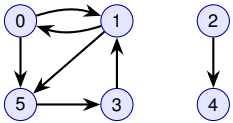
Warshall (1962) dá outra resposta, só com a matriz de adjacência e três laços aninhados: $\Theta(n^3)$, competitivo em grafos densos ($m \approx n^2$) e com um código de quatro linhas. A ideia é de programação dinâmica, e a mesma ideia, com outras operações, dará os caminhos mínimos de Floyd.

**Para aprofundar:** Cormen et al. (2009), §25.2, “Transitive closure of a directed graph” [1]; Warshall (1962) [9].

# Fecho transitivo: definição

## Definição

O **fecho transitivo** de um digrafo $G$ com vértices $0, \ldots, n-1$ é a matriz $R$ com $R[i][j] = 1$ se existe um caminho (com pelo menos um arco) de $i$ até $j$ em $G$, e $R[i][j] = 0$ caso contrário.




adjacência $A$

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 0 | 0 | 1 |
| 1 | 1 | 0 | 0 | 0 | 0 | 1 |
| 2 | 0 | 0 | 0 | 0 | 1 | 0 |
| 3 | 0 | 1 | 0 | 0 | 0 | 0 |
| 4 | 0 | 0 | 0 | 0 | 0 | 0 |
| 5 | 0 | 0 | 0 | 1 | 0 | 0 |

fecho $R$

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 0 | 1 | 0 | 1 |
| 1 | 1 | 1 | 0 | 1 | 0 | 1 |
| 2 | 0 | 0 | 0 | 0 | 1 | 0 |
| 3 | 1 | 1 | 0 | 1 | 0 | 1 |
| 4 | 0 | 0 | 0 | 0 | 0 | 0 |
| 5 | 1 | 1 | 0 | 1 | 0 | 1 |

Em laranja, os pares alcançáveis que não são arcos. A diagonal $R[i][i] = 1$ diz que $i$ está num ciclo: $0, 1, 3, 5$ formam uma componente forte.

**Para aprofundar:** Levitin (2012), §8.4 [7]; grafo `0-1 0-5 1-0 1-5 2-4 3-1 5-3` das aulas de grafos (Feofiloff, 2020) [4].

# Recapitulando: fecho transitivo

## O que aprendemos

- O fecho transitivo responde “existe caminho de $i$ até $j$?” para os $n^2$ pares.
- É uma matriz $n \times n$ de bits; a diagonal marca os vértices que estão em ciclos.
- Com uma busca por vértice, custa $O(n(n+m))$; no grafo de exemplo, $\{0, 1, 3, 5\}$ se alcançam mutuamente e 2 só alcança 4.
- Falta um algoritmo que trabalhe direto na matriz: é o de Warshall.

**Para aprofundar:** Cormen et al. (2009), §25.2 [1].

# O algoritmo de Warshall

# Qual é o subproblema?

## Problema:

Um caminho de $i$ até $j$ pode passar por qualquer vértice. Que família *pequena* de subproblemas do mesmo tipo vai crescendo até o problema todo?

"Caminhos com até $\ell$ arcos" funciona, mas cada passo é um produto de matrizes: $\Theta(n^3)$ por passo, $\Theta(n^3 \log n)$ no total (Cormen, §25.1).

## Solução:

Restringir os vértices **intermediários**: só podem ser usados, no meio do caminho, os vértices $0, 1, \ldots, k-1$. Começa com nenhum ($k = 0$, só arcos diretos) e libera um vértice por rodada.




**Para aprofundar:** Cormen et al. (2009), §§25.1–25.2 [1]; Levitin (2012), §8.4 [7].

# Warshall: definição do subproblema

## Definição

Para $k = 0, 1, \ldots, n$, seja $R^{(k)}[i][j] = 1$ se existe caminho de $i$ até $j$ cujos vértices **intermediários** estão todos em $\{0, \ldots, k-1\}$, e 0 caso contrário.

- $R^{(0)} = A$: sem intermediários, só os arcos.
- $R^{(n)} = R$: todos os vértices liberados, é o fecho transitivo.

## Intuição

A rodada $k$ responde: "agora que o vértice $k$ pode ser usado como escala, quais novos pares ficam ligados?". Como numa malha aérea em que cada rodada abre um novo aeroporto para conexões.

**Para aprofundar:** Warshall (1962) [9]; Cormen et al. (2009), §25.2, "Transitive closure" [1]. Usamos vértices $0, \ldots, n-1$, como em C; o Cormen numera de 1 a $n$.

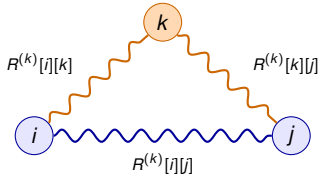
# A recorrência de Warshall

Considere um caminho de $i$ até $j$ com intermediários em $\{0, \ldots, k\}$. Há dois casos:

- **não passa por** $k$: já existia na rodada anterior;
- **passa por** $k$: podemos supor que passa *uma vez só* (cortamos o ciclo); então ele é um caminho $i \rightsquigarrow k$ seguido de $k \rightsquigarrow j$, os dois com intermediários em $\{0, \ldots, k-1\}$.

$$R^{(k+1)}[i][j] = R^{(k)}[i][j] \ \vee \ \left(R^{(k)}[i][k] \wedge R^{(k)}[k][j]\right)$$

Cada célula depende de **três** células da matriz anterior.




**Para aprofundar:** Cormen et al. (2009), §25.2, “Transitive closure” [1]; Levitin (2012), §8.4, algoritmo de Warshall [7].

# Exemplo: Warshall no grafo das aulas de grafos

$R^{(0)} = A$

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 0 | 0 | 1 |
| 1 | 1 | 0 | 0 | 0 | 0 | 1 |
| 2 | 0 | 0 | 0 | 0 | 1 | 0 |
| 3 | 0 | 1 | 0 | 0 | 0 | 0 |
| 4 | 0 | 0 | 0 | 0 | 0 | 0 |
| 5 | 0 | 0 | 0 | 1 | 0 | 0 |

$R^{(2)}$: libera $0, 1$

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 0 | 0 | 0 | 1 |
| 1 | 1 | 1 | 0 | 0 | 0 | 1 |
| 2 | 0 | 0 | 0 | 0 | 1 | 0 |
| 3 | 1 | 1 | 0 | 0 | 0 | 1 |
| 4 | 0 | 0 | 0 | 0 | 0 | 0 |
| 5 | 0 | 0 | 0 | 1 | 0 | 0 |

$R^{(4)}$: libera $2, 3$

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 0 | 0 | 0 | 1 |
| 1 | 1 | 1 | 0 | 0 | 0 | 1 |
| 2 | 0 | 0 | 0 | 0 | 1 | 0 |
| 3 | 1 | 1 | 0 | 0 | 0 | 1 |
| 4 | 0 | 0 | 0 | 0 | 0 | 0 |
| 5 | 1 | 1 | 0 | 1 | 0 | 1 |

$R^{(6)} = R$: libera $4, 5$

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 0 | 1 | 0 | 1 |
| 1 | 1 | 1 | 0 | 1 | 0 | 1 |
| 2 | 0 | 0 | 0 | 0 | 1 | 0 |
| 3 | 1 | 1 | 0 | 1 | 0 | 1 |
| 4 | 0 | 0 | 0 | 0 | 0 | 0 |
| 5 | 1 | 1 | 0 | 1 | 0 | 1 |

- Libera **0**: $1 \to 0 \to 1$ dá $R[1][1]$. Libera **1**: $0 \to 1 \to 0$, $3 \to 1 \to 0$ e $3 \to 1 \to 5$.
- Libera **2**: nada muda, ninguém chega em 2. Libera **3**: $5 \to 3$ herda a linha 3 inteira.
- Libera **4**: nada, 4 não tem saída. Libera **5**: quem chega em 5 (linhas $0, 1, 3$) passa a chegar em 3.

**Para aprofundar:** Levitin (2012), §8.4 (mesmo procedimento, outro grafo) [7].

# Warshall em C

```c
/* r[][] comeca com a matriz de adjacencia e
   termina com o fecho transitivo */
void warshall( int n) {
   for (int k = 0; k < n; ++k)         /* libera k */
      for (int i = 0; i < n; ++i)
         if (r[i][k])                  /* i chega em k */
            for (int j = 0; j < n; ++j)
               if (r[k][j])            /* k chega em j */
                  r[i][j] = 1;
}
```

**Uma matriz só basta.** A recorrência pede $R^{(k)}$ e $R^{(k+1)}$ separadas, mas na rodada $k$ a linha $k$ e a coluna $k$ não mudam: $R[k][j] \vee (R[k][k] \wedge R[k][j]) = R[k][j]$. Atualizar no lugar não altera o que ainda vai ser lido.

Tempo $\Theta(n^3)$, espaço $\Theta(n^2)$. Com bits empacotados em palavras de 64 bits, o laço em $j$ vira um `OU` de linhas inteiras: $\Theta(n^3/64)$.

**Para aprofundar:** Warshall (1962) [9]; Cormen et al. (2009), §25.2, TRANSITIVE-CLOSURE [1]; Levitin (2012), §8.4 [7].

# Recapitulando: Warshall

## O que aprendemos

- Subproblema: $R^{(k)}[i][j]$, caminhos com intermediários em $\{0, \ldots, k-1\}$.
- Recorrência: ou o caminho evita $k$, ou passa por $k$ uma vez só, $i \rightsquigarrow k \rightsquigarrow j$.
- $n$ rodadas, cada uma atualizando $n^2$ células: $\Theta(n^3)$ com uma única matriz.
- No grafo das aulas de grafos, o fecho mostra a componente forte $\{0, 1, 3, 5\}$.

**Para aprofundar:** Cormen et al. (2009), §25.2 [1]; Levitin (2012), §8.4 [7].

# O algoritmo de Floyd

# Por que caminhos mínimos entre todos os pares?

## Problema:

Agora os arcos têm pesos (distâncias, tempos, custos) e queremos a **distância** $\delta(i, j)$ para todos os pares: a tabela de distâncias de um mapa rodoviário, o roteamento numa rede, o diâmetro de um grafo.

Rodar Dijkstra a partir de cada vértice exige pesos **não negativos**; Bellman-Ford a partir de cada vértice aceita negativos, mas custa $O(n^2 m)$, até $O(n^4)$.

## Solução:

O algoritmo de Floyd (1962) é a recorrência de Warshall com $\min$ no lugar de $\vee$ e $+$ no lugar de $\wedge$:

- $\Theta(n^3)$, independente do número de arcos;
- aceita pesos negativos, desde que não haja ciclo negativo;
- e ainda *detecta* ciclos negativos.

**Para aprofundar:** Floyd (1962) [5]; Cormen et al. (2009), introdução do Cap. 25 [1]; Dasgupta et al. (2008), §6.6 [3].

# Floyd: definição do subproblema

## Definição

Seja $G$ um digrafo com pesos $w(i,j)$ e **sem ciclos negativos**. Para $k = 0, \ldots, n$, $d_{ij}^{(k)}$ é o menor peso de um caminho de $i$ até $j$ com intermediários em $\{0, \ldots, k-1\}$ ($\infty$ se não há nenhum). Começamos com

$$d_{ij}^{(0)} = \begin{cases} 0 & i = j, \\ w(i,j) & (i,j) \text{ é arco}, \\ \infty & \text{caso contrário}, \end{cases} \qquad \text{e a resposta é } d_{ij}^{(n)} = \delta(i,j).$$

## Intuição

É o mesmo $R^{(k)}$ de Warshall, só que cada célula guarda o *melhor* caminho com aquelas escalas, e não apenas se ele existe.

**Para aprofundar:** Cormen et al. (2009), §25.2, equação (25.5) [1]; Dasgupta et al. (2008), §6.6 [3].

# Subestrutura ótima e recorrência

Seja $p$ um caminho mínimo de $i$ até $j$ com intermediários em $\{0, \ldots, k\}$. Sem ciclos negativos, podemos tomar $p$ simples. Então:

- se $k \notin p$: $p$ é ótimo para $\{0, \ldots, k-1\}$;
- se $k \in p$: $p = i \rightsquigarrow k \rightsquigarrow j$, e cada pedaço é **mínimo** com intermediários em $\{0, \ldots, k-1\}$ (subcaminhos de caminhos mínimos são mínimos, Cormen Lema 24.1).

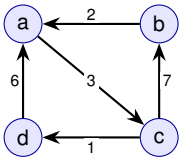
$$d_{ij}^{(k+1)} = \min\left(d_{ij}^{(k)},\ d_{ik}^{(k)} + d_{kj}^{(k)}\right)$$

**Por que isso não vale para caminho *mais longo*?**

Na aula de memoization vimos o contraexemplo: os pedaços de um caminho simples mais longo não são independentes, eles disputam os mesmos vértices. Para caminhos mínimos, cortar ciclos nunca piora (sem ciclos negativos), e os pedaços $i \rightsquigarrow k$ e $k \rightsquigarrow j$ podem ser escolhidos separadamente.

**Para aprofundar:** Cormen et al. (2009), §25.2, equação (25.5), Lema 24.1 e §15.3, Figura 15.6 [1].

# Exemplo: as primeiras rodadas




$D^{(0)} = W$

| | a | b | c | d |
|---|---|---|---|---|
| a | 0 | ∞ | 3 | ∞ |
| b | 2 | 0 | ∞ | ∞ |
| c | ∞ | 7 | 0 | 1 |
| d | 6 | ∞ | ∞ | 0 |

libera $a$

| | a | b | c | d |
|---|---|---|---|---|
| a | 0 | ∞ | 3 | ∞ |
| b | 2 | 0 | 5 | ∞ |
| c | ∞ | 7 | 0 | 1 |
| d | 6 | ∞ | 9 | 0 |

libera $b$

| | a | b | c | d |
|---|---|---|---|---|
| a | 0 | ∞ | 3 | ∞ |
| b | 2 | 0 | 5 | ∞ |
| c | 9 | 7 | 0 | 1 |
| d | 6 | ∞ | 9 | 0 |

- Libera $a$: $d_{bc} = \min(\infty,\ d_{ba} + d_{ac}) = 2 + 3 = 5$ e $d_{dc} = 6 + 3 = 9$.
- Libera $b$: $d_{ca} = \min(\infty,\ d_{cb} + d_{ba}) = 7 + 2 = 9$.

**Para aprofundar:** Levitin (2012), §8.4, exemplo do algoritmo de Floyd [7] (em C, $a$, $b$, $c$, $d$ são os vértices 0, 1, 2, 3).

# Exemplo: as últimas rodadas

libera $c$

| | a | b | c | d |
|---|---|---|---|---|
| a | 0 | 10 | 3 | 4 |
| b | 2 | 0 | 5 | 6 |
| c | 9 | 7 | 0 | 1 |
| d | 6 | 16 | 9 | 0 |

libera $d$: $D^{(4)}$

| | a | b | c | d |
|---|---|---|---|---|
| a | 0 | 10 | 3 | 4 |
| b | 2 | 0 | 5 | 6 |
| c | 7 | 7 | 0 | 1 |
| d | 6 | 16 | 9 | 0 |

Libera $c$, o vértice mais "central": quatro células melhoram, por exemplo

$$d_{ab} = d_{ac} + d_{cb} = 3 + 7 = 10, \qquad d_{db} = 9 + 7 = 16.$$

Libera $d$: $d_{ca}$ cai de 9 (via $b$) para

$$d_{cd} + d_{da} = 1 + 6 = 7.$$

Uma célula pode melhorar mais de uma vez: $d_{ca}$ foi $\infty \to 9 \to 7$.

**Para aprofundar:** Levitin (2012), §8.4, exemplo do algoritmo de Floyd [7].

# Floyd em C

```
#define INF INT_MAX     /* sem arco / sem caminho */

/* d[][] comeca com D(0) e termina com as distancias */
void floyd( int n) {
   for (int k = 0; k < n; ++k)              /* libera k */
      for (int i = 0; i < n; ++i) {
         if (d[i][k] == INF) continue;      /* i nao chega em k */
         for (int j = 0; j < n; ++j)
            if (d[k][j] != INF && d[i][k] + d[k][j] < d[i][j])
               d[i][j] = d[i][k] + d[k][j];
      }
}
```

O laço é o de `warshall`, trocando "`if (r[i][k])`" por "$d_{ik}$ finito" e a atribuição de 1 por um `min`.

**Cuidado com o $\infty$:** `INF + w` estoura o `int`, e com $w < 0$ "$\infty - 5$" parece um caminho. Por isso testamos as duas parcelas antes de somar.

Também basta uma matriz: na rodada $k$, $d_{ik}$ e $d_{kj}$ não mudam porque $d_{kk} = 0$ (Cormen, Exercício 25.2-4).

**Para aprofundar:** Floyd (1962) [5]; Cormen et al. (2009), §25.2, FLOYD-WARSHALL e Exercício 25.2-4 [1].

# Recapitulando: Floyd

## O que aprendemos

- $d_{ij}^{(k)}$: menor peso com intermediários em $\{0, \ldots, k-1\}$; $d_{ij}^{(n)} = \delta(i, j)$.
- Recorrência: $d_{ij}^{(k+1)} = \min(d_{ij}^{(k)},\ d_{ik}^{(k)} + d_{kj}^{(k)})$, justificada pela subestrutura ótima dos caminhos mínimos.
- $\Theta(n^3)$ de tempo, $\Theta(n^2)$ de espaço, uma única matriz.
- No exemplo do Levitin, $d_{ca}$ passou por $\infty$, 9 e 7 até a distância.

**Para aprofundar:** Cormen et al. (2009), §25.2 [1]; Levitin (2012), §8.4 [7].

# Reconstrução, pesos negativos e ordem dos laços

# Por que só a distância não basta?

## Problema:

Um GPS precisa da *rota*, não só do número de quilômetros. E duas perguntas ficaram em aberto:

- o que acontece se houver um ciclo de peso negativo?
- a ordem dos três laços importa?

## Solução:

Como na LCS, guardar a **decisão** junto do valor: uma matriz $prox[i][j]$ com o primeiro vértice depois de $i$ no melhor caminho até $j$.

Ciclos negativos aparecem como $d_{ii} < 0$ na diagonal. E a ordem importa: o laço de $k$ precisa ser o **externo**.

**Para aprofundar:** Cormen et al. (2009), §25.2, “Constructing a shortest path” e Exercício 25.2-6 [1].

# Reconstruindo caminhos com a matriz *prox*

```c
/* inicio: prox[i][j] = j se (i,j) e arco, -1 senao */
/* dentro do floyd, quando melhora pelo vertice k:  */
      d[i][j] = d[i][k] + d[k][j];
      prox[i][j] = prox[i][k];   /* sai como para k */

/* imprime um caminho minimo de i ate j */
void caminho( int i, int j) {
   if (prox[i][j] == -1) { printf( "sem caminho\n"); return; }
   printf( "%d", i);
   while (i != j) {
      i = prox[i][j];
      printf( " -> %d", i);
   }
   printf( "\n");
}
```

No exemplo do Levitin, ao final:

$$prox[b][d] = a, \quad prox[a][d] = c,$$

$$prox[c][d] = d,$$

então o caminho de $b$ até $d$ é

$$b \xrightarrow{2} a \xrightarrow{3} c \xrightarrow{1} d, \quad \text{peso } 6 = d_{bd}.$$

Uma matriz $n \times n$ a mais guarda *todos* os $n^2$ caminhos mínimos; cada um sai em tempo proporcional ao número de arcos.

O Cormen usa a matriz de *predecessores* $\Pi$, que reconstrói de trás para frente; *prox* é a mesma ideia de frente para trás.

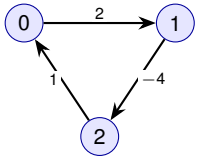
**Para aprofundar:** Cormen et al. (2009), §25.2, equações (25.6)–(25.7) e Figura 25.4 [1].

# Pesos negativos e ciclos negativos

**Pesos negativos são permitidos.** No exemplo do Cormen (Figura 25.4), com arcos de peso $-4$ e $-5$, Floyd obtém, entre outras, $\delta(1,3) = -3$. Dijkstra erraria esse grafo.

**Ciclos negativos não.** Se $C$ é um ciclo de peso negativo, dar mais uma volta sempre diminui o peso: a "distância" não existe. Mas Floyd percebe:

- $d_{ii}$ começa em 0 e só diminui;
- quando o último vértice de $C$ é liberado, $d_{ii} < 0$ para todo $i \in C$;
- basta olhar a diagonal ao final: $\exists\, i : d_{ii} < 0$.




ciclo de peso $2 - 4 + 1 = -1$:
ao final, $d_{00} = d_{11} = -1$ e $d_{22} = -2$
(valores sem sentido, mas negativos)

**Para aprofundar:** Cormen et al. (2009), §25.2, Figura 25.4 e Exercício 25.2-6 [1].

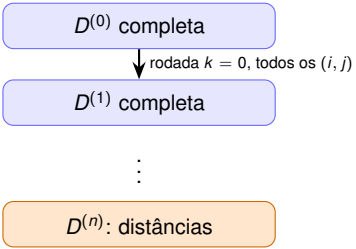
# A ordem dos laços importa

Um erro comum é escrever os laços na ordem "natural" `i`, `j`, `k`, com o $k$ por dentro:

$$d_{ij} \leftarrow \min_k (d_{ik} + d_{kj}).$$

Isso calcula cada $d_{ij}$ **uma vez só**, olhando pares de pedaços que podem ainda não estar prontos. Não é a recorrência de Floyd.

Na recorrência, $d^{(k+1)}$ depende de $d^{(k)}$ *inteira*: a rodada $k$ tem de terminar para todos os pares $(i, j)$ antes de liberar $k + 1$. Por isso $k$ é o laço externo. É a mesma regra das aulas anteriores: preencher a tabela respeitando as dependências.




No tutorial, o Exercício 4 pede um grafo pequeno em que a ordem errada dá a resposta errada.

**Para aprofundar:** Cormen et al. (2009), §25.2, FLOYD-WARSHALL [1]; Dasgupta et al. (2008), §6.6 [3].

# Recapitulando: reconstrução e cuidados

## O que aprendemos

- Uma matriz *prox* atualizada junto com $d$ guarda todos os caminhos mínimos.
- Pesos negativos funcionam; um ciclo negativo deixa $d_{ii} < 0$ na diagonal.
- O laço de $k$ é sempre o externo; "$\infty + w$" precisa ser evitado no código.
- No exemplo do Levitin, o caminho mínimo de $b$ até $d$ é $b\,a\,c\,d$, de peso 6.

**Para aprofundar:** Cormen et al. (2009), §25.2 [1].

# Fechando

# Warshall e Floyd são o mesmo algoritmo

Os dois calculam, para todos os pares, a “melhor combinação” sobre todos os caminhos, trocando só as duas operações:

$$x_{ij} \leftarrow x_{ij} \oplus (x_{ik} \otimes x_{kj})$$

Qualquer par $(\oplus, \otimes)$ com as propriedades de um **semianel** serve. Um exemplo é o **caminho de maior gargalo**: a rota cujo arco mais estreito é o mais largo possível, com $\oplus = \max$ e $\otimes = \min$.

O desafio do tutorial é esse problema.

| problema | $\oplus$ | $\otimes$ | sem arco |
|---|---|---|---|
| fecho (Warshall) | $\vee$ | $\wedge$ | 0 |
| distância (Floyd) | $\min$ | $+$ | $\infty$ |
| gargalo | $\max$ | $\min$ | 0 |

Roy (1959) publicou o fecho transitivo antes de Warshall; por isso o algoritmo também aparece como Roy-Warshall ou Roy-Floyd-Warshall.

**Para aprofundar:** Cormen et al. (2009), §25.2 e notas do Cap. 25 [1]; Roy (1959) [8].

# Qual algoritmo usar para todos os pares?

| algoritmo | pesos | tempo | melhor quando |
|---|---|---|---|
| $n\times$ BFS/DFS | sem pesos | $O(n(n+m))$ | alcançabilidade, grafo esparso |
| Warshall | sem pesos | $\Theta(n^3)$ | alcançabilidade, grafo denso |
| $n\times$ Dijkstra (heap) | $w \geq 0$ | $O(nm \log n)$ | esparso, sem negativos |
| Floyd | sem ciclo negativo | $\Theta(n^3)$ | denso, ou código simples |
| produto de matrizes | sem ciclo negativo | $\Theta(n^3 \log n)$ | (só pela ideia) |
| Johnson | sem ciclo negativo | $O(nm \log n)$ | esparso, com negativos |

Com $m \approx n^2$ arcos, $n\times$ Dijkstra custa $O(n^3 \log n)$: Floyd ganha, e ganha também pela constante pequena e pelo acesso sequencial à memória.

**Para aprofundar:** Cormen et al. (2009), Cap. 25 (§§25.1–25.3) [1]; Johnson (1977) [6].

# Fechando o ciclo da programação dinâmica

| | intervalos | mochila 0/1 | LCS | edição | Floyd |
|---|---|---|---|---|---|
| subproblema | $\mathrm{OPT}(j)$ | $\mathrm{OPT}(i, c)$ | $c[i, j]$ | $E[i, j]$ | $d_{ij}^{(k)}$ |
| o que encolhe | intervalos | itens, capac. | prefixos | prefixos | intermediários |
| dependências | 2 | 2 | 3 | 3 | 3 |
| tabela | vetor | $(n+1) \times (W+1)$ | $(m+1) \times (n+1)$ | $(m+1) \times (n+1)$ | $n \times n\ (\times n)$ |
| tempo | $O(n \log n)$ | $\Theta(nW)$ | $\Theta(mn)$ | $\Theta(mn)$ | $\Theta(n^3)$ |

Em Floyd, a tabela "verdadeira" tem três índices, $n \times n \times (n+1)$; a observação de que a rodada $k$ não altera a linha e a coluna $k$ derruba o espaço para $n^2$, como as duas linhas da LCS e o vetor único da mochila.

**Para estudar:** Cormen, §25.2; Levitin, §8.4; Dasgupta, §6.6. A Prova 2 (29/10) cobre a Unidade 2: divisão e conquista (método mestre, par mais próximo, Karatsuba, Strassen) e programação dinâmica.

**Para aprofundar:** Cormen et al. (2009), §25.2 [1]; Levitin (2012), §8.4 [7].

# Referências I




Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
*Introduction to Algorithms*.
MIT Press, 3 edition, 2009.



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
*Algoritmos: Teoria e Prática*.
Elsevier, Rio de Janeiro, 3 edition, 2012.
Tradução da 3. ed. norte-americana. Seção 4.5.



Sanjoy Dasgupta, Christos H. Papadimitriou, and Umesh V. Vazirani.
*Algorithms*.
McGraw-Hill, 2008.
Seção 2.1: Multiplication (o truque de Gauss).



Paulo Feofiloff.
Algoritmos para grafos (em linguagem C).
Material didático online, DCC/IME-USP, 2020.
Capítulos sobre árvores geradoras de custo mínimo e os algoritmos de Prim e de Kruskal. Acesso em 30 out. 2026.



Robert W. Floyd.
Algorithm 97: Shortest path.
*Communications of the ACM*, 5(6):345, 1962.

# Referências II




Donald B. Johnson.
Efficient algorithms for shortest paths in sparse networks.
*Journal of the ACM*, 24(1):1–13, 1977.



Anany Levitin.
*Introduction to the Design and Analysis of Algorithms.*
Pearson, 3 edition, 2012.



Bernard Roy.
Transitivité et connexité.
*Comptes Rendus de l'Académie des Sciences de Paris*, 249:216–218, 1959.



Stephen Warshall.
A theorem on Boolean matrices.
*Journal of the ACM*, 9(1):11–12, 1962.